Video Processing Final Project

Omri Mali : 212047542, Roy Ziman: 327703013

Stage 1: Video Stabilization

Mathematical Formulation and Pipeline Architecture:

For the stabilization module, we implemented a sparse, feature-tracking geometric pipeline based on methods we learned at lecture. The global transformation between consecutive gray frames is modeled as a Partial 2D Affine transform (4 degrees of freedom), capturing movement (dx, dy) and rotation (da).

The process flows sequentially as follows:

1. **Feature extraction**: Feature tracking coordinates are localized via a classical Harris Corner Detector (*cv2.cornerHarris*). Because the thresholded corner response consists of multi-pixel binary blobs rather than individual coordinates, we use connected component analysis to automatically compute each blob's center of mass (*cv2.connectedComponentsWithStats*). We slice the output array with [1:] to discard index 0, which represents the entire black background canvas component, keeping only structural tracking points.
2. **Sparse optical flow**: Features are tracked from frame t to frame $t + 1$ using the Pyramidal Lucas-Kanade gradient estimation method (*cv2.calcOpticalFlowPyrLK*). This maps local, individual spatial displacement vectors independently across successive frames.
3. **Outlier rejection and global alignment**: A robust Random Sample Consensus (RANSAC) estimator fits the optimal coordinate alignment matrix while forcefully ignoring dynamic outliers, such as the walking subject. It uses the independent point vectors calculated by Lucas-Kanade into a single, comprehensive 4-DOF global transformation matrix ($cv2.estimateAffinePartial2D$) based on consensus majorities .

Systematic Troubleshooting and Optimization Path:

We started with a baseline frame to frame tracking pipeline with standard moving average trajectory smoothing, but we found the output video suffered from drift and border shaking. We resolved these artifacts by making the following optimizations:

1. **Grid distribution:** We noticed that corners near the frame margins shook more than the center. This occurred because the walking human corrupted central features, leaving fewer static reference points at the boundaries. To fix this, we implemented a central spatial mask to completely ignore the subject and enforced a strict 4×4 spatial grid layout to guarantee an even density of tracking points along the absolute outer boundaries.
2. **Sub-pixel refinement:** We have noticed there were some rigid jumps between specific frames, which we believe was the result of rounding errors of tracking the corner centroids. We have made a sub-pixel optimization (cv2.cornerSubPix) immediately after Harris detection, refining coordinates to fractional precision via local image gradient convergence. This smoothed out the rounding errors and eliminated the rigid jumps entirely.
3. **Replacing trajectory smoothing with stationary lock:** The most critical breakthrough came from analyzing the physical nature of the video. Standard moving average trajectory smoothing is built for an intentionally panning or walking camera. Because this sequence was captured from a fixed, handheld position meant to be static, smoothing forced the coordinates to drag along with the slow wandering of the hand. We have changed the architecture to accumulate all step transformations directly back to Frame 0, creating a perfect stationary lock.

<u>Explanation of Advanced Concepts:</u>

1. **Pyramidal Lucas-Kanade Optical Flow:** The classical Lucas-Kanade method relies on a local spatial gradient intensity window, which breaks down if camera movement is larger than the local window size. To solve this, a Pyramid is automatically constructed. We chose a Gaussian pyramid and not a regular down sampling pyramid (as learned at lecture) since a Gaussian pyramid focuses on central pixels and handles noise better.
2. **Sub-Pixel Coordinates:** Standard image operations output feature coordinates as discrete integers (for example pixel row 120, column 340). However, physical camera jitters and optical flow shifts are continuous, real-valued phenomena. By utilizing sub-pixel refinement, the algorithm evaluates the structural dot products of the local neighborhood gradients around a detected corner centroid.

It solves an optimization loop to find the true mathematical sub-pixel peak (For example, [120.34, 340.12]), preventing discrete pixel rounding errors from accumulating into sudden visual skips.

3. **Full Affine vs. Partial Affine vs. Homography:** During our development, we evaluated several geometric transformation models to find the truest fit for the scene:
    a. **Partial Affine (4 DOF):** Assumes rigid transitions: translation, rotation, and uniform scaling. It forces the frame to treat the background as a flat sheet of paper.
    b. **Full Affine (6 DOF):** Adds independent stretching/scaling along the X and Y axes, along with shearing (skew).
    c. **Homography (8 DOF / Projective):** Maps true perspective distortions, allowing the outer corners of a plane to warp at a faster relative speed than its center (accounting for 3D camera pan and tilt).

    Since the handheld video was shot with minimal out of plane tilting, upgrading to a Homography model did not yield a visible change. The simpler Partial Affine model, with the optimization of absolute stationary accumulation and sub-pixel alignment, proved to be the most stable, efficient, and robust mathematical choice.

Stage 2: Background Subtraction

Mathematical Formulation and Pipeline Architecture:

The second stage receives the stabilized video from Stage 1 and separates the walking person from the static scene. In the course lectures, background subtraction was introduced as a per-pixel temporal decision problem: for each pixel location (x, y), we observe its intensity profile across time and decide whether the current observation belongs to the static background or to a dynamic foreground object. The simplest approach would be frame differencing, but that is too sensitive to noise and residual stabilization motion. Therefore, we used a Gaussian Mixture Model background model, implemented through OpenCV MOG2, and then added our own temporal consistency, component selection, and forward/backward mask fusion around it.

For every pixel value x_t at time t, the background model is represented as a mixture of K Gaussian distributions. In our implementation x_t is not a raw BGR pixel: before modeling, each frame is converted to HSV and blurred slightly, so that the model is less sensitive to small illumination noise and small stabilization jitter.

The implemented pipeline is organized as follows:

1. **Storing the Input from Stage 1:** the stabilized video is read into memory. Since our method compares two temporal directions, the full frame list is needed.
2. **Forward modeling:** MOG2 is trained on the frames in chronological order. Each frame is converted to HSV, blurred with a small Gaussian kernel, and passed to the model with a small learning rate.
3. **Forward extraction:** the trained model is frozen using learningRate = 0.0. Each frame is classified into background, shadow, or foreground. Only true foreground pixels with value 255 are retained.
4. **Backward modeling and extraction:** the exact same process is repeated on the reversed frame order. After extraction, the resulting masks are reversed back to the original order.
5. **Morphological cleanup:** opening removes isolated noise, closing reconnects fragmented body regions, and contour-based hole filling removes black gaps inside the silhouette.

6. **Person component selection:** connected components are extracted and scored according to area, height, distance from the previous center, and bounding-box consistency.
7. **Forward/backward fusion:** the two masks are combined. If they overlap, the overlap is treated as reliable support. If they do not overlap, the smaller mask is used to reduce background leakage.

Systematic Troubleshooting and Optimization Path:

We started with a direct MOG2 background subtractor. This produced ghosting and background leakage, especially near high-contrast posters and at the end of the sequence. The reason is that background subtraction is extremely sensitive to residual stabilization motion: a wall that moves by only a few pixels may be classified as foreground. In addition, if the person appears over the same background location for many frames, the background model can partially absorb the person or leave a ghost when the person moves away.

The following optimizations were introduced during development:

1. **HSV preprocessing:** Instead of applying MOG2 directly on BGR frames, frames are converted to HSV. HSV partially separates chromatic content from brightness and produced more stable masks under small illumination changes.
2. **Gaussian blur before modeling:** A 5×5 blur suppresses single-pixel jitter and camera-stabilization noise before the model observes the frame.
3. **Bidirectional processing:** A forward-only model is weaker at the end of the video, while a backward-only model is weaker at the beginning. Combining both directions reduces this temporal bias.
4. **Shadow suppression:** MOG2 can output 127 for shadows. We discard it and keep only 255, because shadows created false foreground regions connected to the feet and floor.
5. **Temporal component tracking:** Instead of selecting the largest component blindly, every connected component is scored using its similarity to the previous person position and size. This prevents the algorithm from jumping to a wall or poster artifact.

6. **Silhouette solidification:** Since background subtraction often creates holes inside dark clothing, the selected person component is closed and filled after it has already been selected.

Rejected alternatives: KNN background subtraction and GrabCut refinement were tested. KNN did not improve the final quality, and GrabCut increased runtime significantly without solving the main leakage problem. We therefore kept MOG2 with our custom temporal logic.

Explanation of Advance Concepts:

1. ***cv2.createBackgroundSubtractorMOG2:*** This function creates an online Gaussian Mixture Model background subtractor. Internally, each pixel maintains several Gaussian modes. A new observation is compared to the existing modes. If it is close enough to one mode, that mode is updated. Otherwise, a new mode is created or an old weak mode is replaced. The library function only produces a raw foreground mask.
2. **cv2.*morphologyEx:*** Morphological operations modify the binary mask according to a small structuring element. Opening is erosion followed by dilation: for removing isolated white noise. Closing is dilation followed by erosion: for reconnecting nearby white regions and filling small gaps. We use elliptical kernels because the human body is not rectangular, and elliptical kernels expand the silhouette more naturally around arms, legs, and torso.
3. ***cv2.findContours:*** Contours describe the boundary curves of connected white regions in a binary image. *RETR_CCOMP* returns a two-level hierarchy: external object boundaries and internal holes. We use this hierarchy in *_fill_holes*: if a contour has a parent, it is an internal hole, and we draw it in white. *RETR_EXTERNAL* is used later in *_solidify* because at that point, we only want the outer boundary of the selected person and not the inner holes.
4. ***cv2.connectedComponentsWithStats:*** After the mask is cleaned, it may still contain several disconnected foreground blobs. This function labels each connected white region and returns its area, bounding box, and centroid. We use those geometric measurements to choose the component most likely to be the

person. The scoring function rewards large and tall components and penalizes sudden jumps in location, height, and width compared to the previous frame:

$$score = area + 70 \cdot height - 450 \cdot distance - 25000 \cdot \Delta height - 12000 \cdot width$$

5. **Forward/backward mask fusion:** The algorithm computes one mask from the forward pass and one from the backward pass. If they overlap, the overlap is considered reliable foreground support. This preserves body pixels that appear in only one pass while rejecting distant leakage. If the masks do not overlap, we choose the smaller mask, because large masks usually indicate background contamination.
6. **Border suppression:** The stabilized video may contain unstable pixels at the image boundaries due to warping (which is performed at the stabilization stage). these border pixels frequently appear as false motion. Therefore, before connected component analysis, we explicitly set a fixed margin around the frame to zero.

Stage 3: Video Matting

Mathematical Formulation and Pipeline Architecture:

The matting module acts as the compositing engine of the project, taking the extracted color foreground from Stage 2 and seamlessly blending them onto a new static background image. Instead of using a harsh binary cut, the pipeline generates a continuous alpha matte $\alpha \in [0,1]$ to achieve smooth edge blending.

The process flows sequentially as follows:

1. **Background Realignment**: The target background image is loaded and dynamically resized using bilinear interpolation (cv2.resize) to match the exact spatial dimensions (width and height) of the video frames.
2. **Alpha Generation via Distance Transforms**: To create a smooth transition zone along the silhouette boundaries, the binary mask is passed through a dual distance transform process (cv2.distanceTransform). The algorithm calculates the Euclidean (L_2) distance from every pixel to the nearest zero pixel (inside the foreground) and the nearest non-zero pixel (outside the foreground). The normalized alpha value for each pixel is mathematically formulated as:

$$\alpha = \frac{dist_{in}}{dist_{in} + dist_{out} + 10^{-6}}$$

3. **Edge Feathering**: The continuous alpha map is smoothed using a Gaussian blur kernel proportional to a specified feathering radius to eradicate aliasing artifacts.
4. **Alpha Blending (Compositing)**: The final composite frame is constructed using a vectorized linear blending equation applied independently across the 3 color channels:

$$I_{matted} = \alpha \cdot I_{FG} + (1.0 - \alpha) \cdot I_{BG}$$

The resulting array is clipped to the valid $[0,255]$ range and cast back to unsigned 8-bit integers.

Explanation of Advance Concepts:

1. ***Dual Distance Transform Optimization:*** Instead of relying on standard morphology (like dilation or erosion, cv2.distanceTransform measures accurate distances. By calculating the ratio of the internal distance (dist_in) to the total distance boundary field (dist_in + dist_out), the pipeline maps a smooth, gradual gradient right across the contour edges. The 10^(-6) epsilon value prevents division-by-zero errors in fully static background regions.
2. ***Vectorized Alpha Compositing:*** Alpha matting scales linearly with frame count and resolution. Rather than processing pixels individually, the blending operation leverages NumPy's internal C-array vectorization. The equation $\alpha \cdot FG + (1-\alpha) \cdot BG$ structurally cross-fades the matrix properties simultaneously across all color channels, keeping the processing time per frame exceptionally low.

Stage 4: Object Tracking via Particle Filtering

Mathematical Formulation and Pipeline Architecture:

For the tracking module, we built a Particle Filter designed to track the moving person over time. We implement this like we learned in class.

The tracking state vector at any given frame is defined as a 6-dimensional array

$$s = [x_c, y_c, w_h, h_h, v_x, v_y]$$

Where (x_c, y_c) represents the person bounding box center, (w_h, h_h) represents the half-width and half-height, and (v_x, v_y) represents the horizontal and vertical velocity components.

The algorithm processes frames sequentially through a Prediction-Correction-Resampling loop:

1. **Prediction**: The next generation of $N = 400$ particles is predicted by applying a deterministic velocity drift to the prior state, supplemented by stochastic Gaussian white noise:

$$s_t = s_{t-1} + \begin{bmatrix} x \\ 0 \\ 0 \\ 0 \\ v_x \\ 0 \end{bmatrix} * \mathcal{N}(0, \Sigma)$$

2. **Correction (Measurement Weighting)**: The importance weight W_n of each particle is evaluated by extracting a 3D color histogram ($16 \times 16 \times 16$ bins) from its candidate region (cv2.calcHist). The histogram is normalized and compared against the target reference histogram q (initialized at Frame 1) using the Bhattacharyya distance metric:

$$\rho(p, q) = \sum_i \sqrt{p(i) \cdot q(i)}$$

$$W_n = \exp(20.0 \cdot \rho(p, q)))$$

3. **Renormalization**: The weights are normalized so they sum to 1.0, and an accumulated Cumulative Distribution Function (CDF) map is generated.

4. **Systematic Resampling**: particles are sampled from the current set based on their importance weights using a uniform distribution look-up inversion (np.searchsorted). The state with the maximum weight coefficient is selected as the optimal target position for that frame.

Systematic Troubleshooting and Optimization Path:

Implementing a particle filter on full-resolution video streams introduces severe computational bottlenecks, tracking challenges and optimization ideas:

1. **High Computational Latency**: Calculating 400 separate 3D color histograms per frame on a full-sized video is quite slow. We resolved this by cutting the resolution in 4 (SCALE_FACTOR = 0.25), it didn't hurt the results because the maximum error of some location of the bounding box is at most 3px for each axis.
2. **Setting Most Of The Changeable Parameters to Zero**: We set the parameters: y (center of the box in the horizontal axis), h (height of the box), w (width of the box) and v_y (the velocity in the horizontal axis) to zero because we saw in the video that the person walk straight, so the only things that can be changed are x (the center of the box in the vertical axis) and v_x (the velocity in the vertical axis).
3. **Setting Bounding Box to Be Smaller for The Particle Filter**: When we run the particle filter in the full-size box (the sizes of the person) it sometimes would ended in a local minima, so by shrinking the size of the bounding box we got consistence results that avoid local minima, we calculate the differences and add it when we plotting the bounding box.

Explanation of Advance Concepts:

1. ***Cv2.calcHist:*** To evaluate how closely a candidate particle matches the target subject, the tracking module extracts a multi-dimensional color fingerprint using cv2.calcHist. The pipeline constructs a 3D Color Histogram across the three color channels simultaneously, quantizing the dense BGR color space into a manageable $16 \times 16 \times 16$ bin configuration (4,096 total combinations).

2. **CDF Resampling via Binary Search:** To replace poorly positioned particles with high-weight successes without introducing an $O(N^2)$ algorithmic slowdown, we map the normalized weights to a cumulative sum array (CDF). Generating N random uniform numbers and utilizing np.searchsorted allows us to perform a binary search index lookup. This handles the entire resampling phase in $O(N \log N)$ time, ensuring highly efficient state tracking across long video files.
3. **Bhattacharyya Distance:** The Bhattacharyya coefficient ρ outputs a value between 0 and 1 indicating color distribution similarity. However, standard linear weights fail to cleanly separate the best-matching particles from mediocre background noise. By mapping the coefficient to an exponential scale ($\exp(20.0 \cdot \rho)$), we dramatically amplify the score variance. Particles that capture the exact color layout of the person receive massive, dominant importance weights, forcing the filter to rapidly converge on the true target.